

Flutter App

Project Structure & Developer Guide

A plain-language walkthrough of how the app is built

This guide explains how the app is organized — where code lives, how screens manage their data, how the app talks to the backend, and how releases are shipped. It's written so that anyone joining the project can find their way around quickly.

1. Environment Configuration

The app runs in three different "environments," chosen by a *flavor* at launch (set in `main.dart`):

Flavor	Environment	Backend URL used
<code>prod</code>	Production	<code>api.example.com</code>
<code>dev</code>	Development	<code>dev-api.example.com</code>
anything else	Test	<code>dev-api.example.com</code>

The selected environment drives `Config.baseUrl`, which decides which server the app connects to.

2. Folder & Architecture Overview

All source code lives under `lib/`, split into a **core** layer (logic) and a **ui** layer (screens & widgets):

```
lib/
├─ main.dart           # Firebase init, flavor detection, setupLocator(env)
├─ app.dart            # GetMaterialApp + ScreenUtilInit root
├─ locator.dart        # GetIt singleton registrations (all services)
├─ core/
│   ├─ config/         # Config class – baseUrl per Env
│   ├─ constants/      # Colors, strings, images, EndPoints, styles
│   ├─ enums/          # ViewState, Env, auth states
│   ├─ models/         # Data models + fromJson/toJson
│   ├─ services/       # All business logic
│   └─ view_model/     # BaseViewModel
└─ ui/
    ├─ screens/        # Feature screens + their providers
    └─ custom_widgets/ # Shared UI components
```

3. How Screens Manage State

Every screen comes in a pair: a `*_screen.dart` file (the visuals) and a co-located `*_provider.dart` file (the logic and data). Screens listen to their provider and rebuild when data changes.

```
// Provider – holds the logic
class FooProvider extends BaseViewModel {
  void doWork() {
    setState(ViewState.busy); // notifies the screen to show loading
    // ... do the work ...
    setState(ViewState.idle); // notifies the screen it's done
  }
}

// Screen – listens and rebuilds
ChangeNotifierProvider(
  create: (_) => FooProvider(),
  child: Consumer<FooProvider>(
    builder: (context, model, child) { ... },
  ),
)
```

Rule of thumb: Always call `setState()` (not `notifyListeners()` directly) for changes that update the view state. `BaseViewModel.setState()` safely skips updates after a screen is disposed, preventing crashes.

4. The Service Layer

All business logic lives in **services**, registered as singletons via GetIt. You can reach any service from anywhere with `locator<ServiceName>()`.

⚠ **Important naming trap:** `DatabaseService` is **NOT** a local database. It is the REST API client — every call to the backend goes through it. There is no SQLite involved.

`DatabaseService` → `ApiService.launchDio()` → `Dio` → **REST API**

`ApiService.launchDio()` builds a fresh Dio instance for each request (not pooled) and reads the `accessToken` from `LocalStorageService` every time.

Service	What it does
<code>DatabaseService</code>	All REST API calls (misnamed — no SQLite)
<code>ApiService</code>	Raw HTTP (get/post/put/delete/multipart), Dio setup, session validation
<code>AuthService</code>	Login/signup/token management; holds <code>User user</code> and <code>Lawn? selectedLawn</code> (the active lawn)
<code>LocalStorageService</code>	SharedPreferences wrapper (keys: <code>accessToken</code> , <code>onboardingCount</code> , <code>appVersion</code>)
<code>SocketService</code>	socket.io WebSocket — <code>connect()</code> / <code>joinRoom()</code> / <code>leaveRoom()</code>
<code>AccountStatusManagerService</code>	Subscription / account state
<code>LocationService</code>	Caches GPS as <code>currentPosition</code> ; <code>getCurrentPosition()</code> refreshes it
<code>PermissionService</code>	Device permission requests (e.g. <code>locationAccess()</code>)
<code>NotificationService</code>	FCM push + local notifications
<code>ConnectivityService</code>	Network connection monitoring

5. Navigation & UI Conventions

Navigation (GetX)

Routing uses GetX: `Get.to()` to push, `Get.back()` to pop, `Get.offAll()` to reset the stack.

UI rules

- **Responsive sizing:** always use `flutter_screenutil` units — `.w`, `.h`, `.sp`, `.r`
- **Text scaling:** fixed at `TextScaler.linear(1.0)` in `app.dart` — do not use `MediaQuery.textScaleFactor`
- **Loading:** wrap the `Scaffold` in `ModalProgressHUD`, driven by `model.state == ViewState.busy`
- **Placeholders:** use `shimmer` package widgets from `custom_widgets/`

6. Logging

Use the project's `CustomLogger` — never `print()` or `debugPrint()` in business logic (`debugPrint` is allowed only as a fallback inside catch blocks).

```
final log = CustomLogger(className: '@MyClass');
log.i('@myMethod: message'); // info
log.d('@myMethod: message'); // debug
log.e('@myMethod: message'); // error
log.w('@myMethod: message'); // warning
```

Convention: class names are prefixed with `@ClassName` , and method names are prefixed with `@` inside messages.

7. App Startup Sequence (SplashScreen)

- 1 Connectivity monitoring starts
- 2 `SizeConfig` initializes
- 3 `LocalStorage.init()` + internet check (in parallel)
- 4 No internet → show `NoInternetScreen`
- 5 Socket connects
- 6 FCM configured (in background)
- 7 Onboarding check (`onBoardingPageCount < 2` → `OnBoardScreen`)
- 8 `auth.initialSetup()` — token check + user fetch
- 9 Route decision → `WelcomeScreen` / `VerifyOtpScreen` / `RootScreen`

8. CI/CD & Git Branch Strategy

Build pipeline

- Push to the `test` branch → Android build + upload to Google Play internal track (`build_apk_appbundle.yml`)
- Version comes from `pubspec.yaml` : `version: MAJOR.MINOR.PATCH+BUILD_NUMBER`
- Feature work uses `feature-branch` naming

Branch flow

dev — active development, open PRs here

test — triggers Android CI on push

prod — production releases

dev → test (CI runs) → prod